

Clang15.0.7适配旧版本MSVC文档

- by黄熙涵、欧阳照林、廖中煜
- 仓库：<https://github.com/wingrew/llvm-project-msvc>
- 时间：2025/7/6起

一、分工

- 黄熙涵：负责统筹，收集资料，环境搭建指导，部分文档梳理，测试集编写
- 欧阳照林：收集资料，部分文档梳理，测试集编写
- 廖中煜：收集资料，部分文档梳理，测试集编写

二、环境搭建

基础平台为：windows
ide:vscode(clang) vs(msvc)

- clang15.0.7安装
- msvc2015安装
- msvc2012安装(由于微软已经将旧版本归档，应用及文档需要从镜像网站获取)

基础编译测试指令： clang-cl -fsyntax-only file.cpp

三、方言梳理

序号	MSVC编译方言 \\特定情况\\宏	clang15.0.7通过情况 (支持/部分支持/ 不支持)	原因	备注
1	内置运算符 (sizeof)	部分支持	使用sizeof时clang紧随类型时必须使用括号，但是msvc不必	https://learn.cn/cpp/cpp/?view=msvc-
2	Microsoft 专用 C++ 关键字 (__wchar_t)	部分支持	如果以 L 为前缀的宽字符文本包含多字符序列，则值取自第一个字符	https://learn.cn/cpp/cpp/?view=msvc-
3	Microsoft 专用 C++ 关键字 (__event)	不支持	不含此关键字	https://learn.cn/cpp/cpp/?view=msvc-140
4	Microsoft 专用 C++ 关键字 (__raise)	不支持	不含此关键字	https://learn.cn/cpp/cpp/?view=msvc-140
5	Microsoft 专用 C++ 关键字 (__hook)	不支持	不含此关键字	https://learn.cn/cpp/cpp/?view=msvc-140
6	Microsoft 专用 C++ 关键字 (__based)	不支持	支持异常，指针识别不到	https://learn.cn/cpp/cpp/?view=msvc-140

序号	MSVC编译方言 \特定情况\宏	clang15.0.7通过情况 (支持/部分支持/ 不支持)	原因	备注
7	头文件 (include)	部分支持	不允许""开头	https://learn.microsoft.com/cpp/cpp/using-preprocessor-directives#using-include
8	选择语句 (switch)	部分支持	在 MS 模式下，我们假设所有枚举都具有明确的底层类型int。 这几乎是我们应该做的，但并非完全是。MS 枚举始终为 4 个字节且默认带符号，但它们没有明确的底层类型，因此这种缩小范围的检查误导	https://learn.microsoft.com/cpp/cpp/using-enumerations#using-enumerations
9	声明和定义 (decltype)	部分支持	Clang 在 decltype 上下文中将 static_cast<Base&>(*this) 视为试图将 Base&（非 const 左值引用）绑定到一个 Derived 对象，而没有正确考虑继承关系。	https://learn.microsoft.com/cpp/cpp/using-qualified-names#using-qualified-names
10	初始值设定项 (list)	部分支持	此处为msvc初始化不严谨	https://learn.microsoft.com/cpp/cpp/initializers#initializers
11	继承 (private)	部分支持	此处为msvc初始化不严谨	https://learn.microsoft.com/cpp/cpp/using-qualified-names#using-qualified-names
12	析构函数	部分支持	此处为msvc初始化不严谨	https://learn.microsoft.com/cpp/cpp/using-qualified-names#using-qualified-names
13	模板	部分支持	具有默认非类型参数的模板类永远不会与具有可变模板参数的函数的返回类型匹配C++的模板参数匹配规则要求模板参数的数量和类型严格匹配。tuple<Args...> 试图将参数包 Args... 直接映射到 tuple<T1, T2>，但 Args... 只提供类型参数（例如），而 T2 是一个非类型参数，编译器无法自动将 Args... 映射到 T1 和 T2 的组合。即使 T2 有默认值 0，C++ 的模板推导不会自动将 tuple 解析为 tuple<int, 0>，因为 Args... 是一个类型参数包，而 T2 是一个非类型参数，二者的性质不同，无法直接匹配。	https://learn.microsoft.com/cpp/cpp/using-qualified-names#using-qualified-names
14	模板	部分支持	模板函数template void f(MUT t)是显式模板函数实例化的可行候选者，但是 clang 没有注意	https://learn.microsoft.com/cpp/cpp/using-qualified-names#using-qualified-names
15	枚举	部分支持	clang 拒绝此代码，因为定义没有提供固定的底层类型，但 MSVC 和 edg w/--microsoft接受它作为扩展错误的核心问题：在 C++ 中，枚举可以分为两种：非作用域枚举（enum）和作用域枚举（enum class 或 enum struct）。C++11 引入了固定底层类型的枚举（通过 : 类型 指定，例如 : int）。如果一个枚举被前向声明为具有固定底层类型（如 enum E : int;），那么后续的定义（enum E {}）必须：使用相同的底层类型（这里是 int）。保持相同的枚举种类（作用域或非作用域）。不能省略底层类型（即不能在定义时不指定底层类型）。	https://learn.microsoft.com/cpp/cpp/using-enumerations#using-enumerations

序号	MSVC编译方言 \特定情况\宏	clang15.0.7通过情况 (支持/部分支持/ 不支持)	原因	备注
			在代码中, enum E {}; 没有指定底层类型, 而之前的声明 enum E : int; 指定了底层类型为 int, 这导致了冲突。	
16	__declspec 修饰符中的 Microsoft 关键字 (noreturn)	部分支持	gcc、clang 和 MSVC 都提供了编译器特定的机制来指示函数是否“无返回值”。gcc 和 clang 有 attribute((noreturn)), 而 MSVC 有 __declspec(noreturn)。gcc 和 MSVC 将其编译器特定的属性视为满足第一个声明属性的要求, 但 clang 却不这么做。	https://learn.microsoft.com/cpp/cpp/using-attributes?view=msvc-170&https://github.com/microsoft/vcpkg/issues/10000
17	__declspec 修饰符中的 Microsoft 关键字 (noreturn)	部分支持	gcc、clang 和 MSVC 都提供了编译器特定的机制来指示函数是否“无返回值”。gcc 和 clang 有 attribute((noreturn)), 而 MSVC 有 __declspec(noreturn)。gcc 和 MSVC 将其编译器特定的属性视为满足第一个声明属性的要求, 但 clang 却不这么做。	https://learn.microsoft.com/cpp/cpp/using-attributes?view=msvc-170&https://github.com/microsoft/vcpkg/issues/10000
18	重载	部分支持	[clang] 在重载解析期间, clang 无法识别从具有 const 限定的 prvalue 到 rvalue ref 的引用绑定, 因为该引用格式不正确由于引用不兼容MyString&&而无法绑定到结果	https://learn.microsoft.com/cpp/cpp/using-attributes?view=msvc-170&https://github.com/microsoft/vcpkg/issues/10000
19	模板	部分支持	Clang 未能考虑在实例化上下文中可到达的默认参数	https://learn.microsoft.com/cpp/cpp/using-attributes?view=msvc-170&https://github.com/microsoft/vcpkg/issues/10000
20	模板	部分支持	[ClangCL] 显式模板实例化与 dllexport 结合的不兼容行为	https://learn.microsoft.com/cpp/cpp/using-attributes?view=msvc-170&https://github.com/microsoft/vcpkg/issues/10000
21	constexpr (C++)	部分支持	[clang-cl]constexpr字段不能引用 DLL 导出的符号	https://learn.microsoft.com/cpp/cpp/using-attributes?view=msvc-170&https://github.com/microsoft/vcpkg/issues/10000
22	显式重写	部分支持	不兼容	https://learn.microsoft.com/cpp/cpp/using-attributes?view=msvc-170&https://github.com/microsoft/vcpkg/issues/10000
23	预处理指令 (import)	不支持	特性不支持	https://learn.microsoft.com/cpp/preprocessor/import-directives?view=msvc-170
24	Variadic 宏	部分支持	语法不一致	https://learn.microsoft.com/cpp/preprocessor/macros?view=msvc-170
25	Pragma指令 (pack杂注)	部分支持	pack-pop的语法不一致	https://learn.microsoft.com/cpp/preprocessor/view=msvc-170
25	Pragma指令 (pack杂注)	部分支持	pack-pop的语法不一致	https://learn.microsoft.com/cpp/preprocessor/view=msvc-170

序号	MSVC编译方言 \\特定情况\\宏	clang15.0.7通过情况 (支持/部分支持/ 不支持)	原因	信息
25	Pragma指令 (init_seg)	部分支持	语法不一致	https://learn.microsoft.com/cpp/preprocessor/view=msvc-
26	Microsoft 特定的属性(all)	不支持	Microsoft 特定的属性 <code>gsl::suppress([, justification:])</code> 是一个字符串，用于指定要隐藏的规则的名称。该 optional justification 字段允许您解释禁用或禁止显示警告的原因。指定选项 <code>/analyze:log:includesuppressed</code> 后，此值将显示在 SARIF 输出中。它的值是 UTF-8 编码的窄字符串文字。	https://learn.microsoft.com/cpp/cpp/view=msvc-

四、测试集

测试集主要依据微软官方文档和部分issue编写，主要目的为语法通过编译。
我们在实践中发现，文档并没有将MSVC所有的专有特性写出，因此采取了从issue中补充的办法，以增加全面性。
目前共有几十个测试用例，来源清晰，依据明确。此外，我们正在收集实际工程项目，进一步测试clang-cl的兼容性。